SectorQ: Highly Scalable Pauli Propagation on HPC Systems with Window-Conserved Parity Sectors

ANONYMOUS AUTHOR(S), Submission under review, Anonymous

This paper presents *SectorQ*, a distributed Pauli propagation system for observable simulation of quantum circuits with more than 100 qubits on GPU clusters. Lower coefficient cutoffs increase the retained expansion and its memory requirements, while gate-level global merges impose coordination costs on distributed GPU propagation. *SectorQ* constructs window-conserved parity sectors from the generators of consecutive gates, assigning all contributions to an equal Pauli string to one sector and permitting local merge and truncation throughout each window. The runtime selects conserved rows using sampled live terms during GPU execution and stores sectors in distributed host memory, with one sector loaded per GPU worker and term exchange restricted to window boundaries. An evaluation of seven circuits on up to 512 NVIDIA A100 GPUs shows 4.4-9.7× speedups over original-circuit *cuPauliProp* at the largest common settings on 16 GPUs and a 140× larger completed term population on 512 GPUs.

1 Introduction

Quantum processors with more than 100 qubits have enabled experiments beyond sampling demonstrations, including measurements of observables in interacting physical models [2, 10]. Observable expectation values characterize quantities such as magnetization and energy in quantum simulation, with applications in physics, chemistry, and materials science [7]. For example, a 127-qubit processor measured the magnetization of a kicked Ising model at a scale beyond direct state-vector simulation [10]. Classical calculations of these observables provide a basis for assessing the accuracy and computational cost of the quantum experiment. The tractable circuit size depends on the classical representation, its approximation, and the computing resources available for the calculation.

Current processors remain noisy intermediate-scale quantum (NISQ) devices, with errors that affect measured expectation values [14]. Error mitigation estimates an observable from repeated measurements, including measurements at amplified noise levels [10]. Classical simulation on high-performance computing (HPC) systems provides an independent reference for evaluating these estimates. Such references support hardware validation, error-mitigation assessment, and algorithm development, subject to the numerical error of the classical method. Tensor network and Pauli propagation studies have reproduced observables from the 127-qubit experiment [3, 18]. These calculations establish the relevance of classical simulation even when a full state vector exceeds available memory.

Three representations are relevant to the classical simulation of these circuits. State vector simulation stores all 2^n complex amplitudes of an n-qubit state, requiring 16 PiB for 50 qubits in double precision [8]. Tensor network simulation represents the state through tensors, with contraction cost determined by the network structure and the required tensor dimensions [18]. Pauli propagation represents an observable as a sum of Pauli strings and propagates it through the circuit in the Heisenberg picture [3, 17]. Its storage and execution costs depend on the number of retained terms as well as the string length and gate sequence. Current NVIDIA and IBM libraries support this representation [13, 15]. Our work focuses on distributed Pauli propagation, where coefficient truncation controls the retained expansion and permits calculations beyond the memory range of full state vectors.

The retained expansion can exceed the capacity of one GPU as the coefficient cutoff decreases. Terms also accumulate between scheduled truncations, requiring temporary storage beyond the

Author's Contact Information: Anonymous Author(s), Submission under review, Anonymous.

Table 1. Features of Pauli propagation systems

System	Method	MN	GPU	LP	RP	HM
PauliPropagation.jl [17]	Pauli propagation, one node					
SPD [16]	Sparse Pauli dynamics		✓			
ORQA [5]	Structured distribution map	✓				
MonoProp [1]	Distributed Pauli propagation	✓				
cuPauliProp [13]	Global merge after every gate	✓	✓			
paulistrings-rs [12]	Fixed linear partition	✓	✓	✓		
***SectorQ* (Ours)**	**Window-conserved parity sectors**	✓	✓	✓	✓	✓

MN denotes multi-node execution, GPU device execution, LP locality-preserving partitions, RP run-time partitions derived from gates and live terms, and HM host storage beyond GPU capacity.

retained expansion. For the 127-qubit kicked Ising circuit, the maximum population at a window boundary increases from 159 million terms at cutoff 10^{-5} to 11.9 billion at 10^{-6}. At 40 bytes per term, these populations require about 6.4 GB and 475 GB before execution buffers. Distributed implementations address this storage requirement on GPU clusters and on CPU systems, including Fugaku runs with 2^{17} processes [5, 13]. However, the distributed GPU implementations still incur coordination costs at individual gates and require their local expansions to fit device memory.

Distributed GPU propagation has two principal constraints. First, terms with the same Pauli string must be merged before coefficient truncation, including contributions produced on different GPUs. The distributed mode of *cuPauliProp* coordinates this merge after every gate [13]. The measured fixed cost increases from 5.9 ms per gate on four GPUs to 29.8 ms on 16 GPUs. Second, device memory must accommodate retained terms, intermediate branches, and merge workspace. An anticommuting rotation can produce two terms from one input, increasing the expansion between truncations. Under these requirements, *cuPauliProp* retains about 100 million terms per GPU with a 36 GB memory budget. The GPU implementation of paulistrings-rs also stores its assigned partitions in device memory [12]. Deferred deduplication reduces global merge frequency but increases storage for duplicate strings. Thus, a distributed design must provide merge locality across multiple gates and accommodate the stored operator beyond the working buffers of individual GPUs.

Previous systems address arithmetic, distribution, and storage at different levels, as summarized in Table 1. Single-node simulators use sparse data structures, circuit symmetries, and GPU kernels to reduce propagation cost [3, 16, 17]. ORQA uses a structured distribution map to limit communication partners, and MonoProp supports distributed CPU propagation [1, 5]. The distributed mode of *cuPauliProp* combines contributions across GPUs after individual gates [13]. The linear partition in paulistrings-rs assigns strings through fixed bit parities, with a predictable communication partner when a generator changes a partition label [12]. Commutator-graph analysis also identifies operator subspaces invariant under a set of generators [19]. None of these implementations combines window-conserved rows, row selection from live terms, and host-memory sector execution.

SectorQ constructs a partition for each window of consecutive gates and selects its rows during execution. The window generators define conserved parities that assign every contribution to an equal Pauli string to the same sector. The runtime selects among these parities using a sample of the live terms, then assigns each sector to one node for the window. This assignment confines merge and truncation to sector execution until the next window boundary. The host-memory store retains the sectors, and each GPU loads one sector at a time with the workspace required by the propagation library. An oversized sector is subdivided using further rows valid for its remaining

operations. Operator capacity consequently depends on distributed host storage and the availability of executable sector subdivisions, while term exchange occurs at window boundaries.

In this paper, we present *SectorQ*, a Pauli propagation system for GPU clusters that partitions the terms by the parities that a window of gates conserves. A rotation gate maps each term to itself and to its product with the generator of the gate, and every parity that the generator leaves unchanged assigns the same value to both terms. Specifically, *SectorQ* introduces 1) window-conserved parity sectors that the system recomputes for every window from the generators of its gates, 2) a run-time choice of the parities and the number of sectors from a sample of the live terms while the GPUs process the current window, and 3) a host-memory sector store from which each GPU loads one sector at a time. With these sectors, the terms move between nodes only at window boundaries, and the number of terms that *SectorQ* holds follows the host memory of all nodes. Our evaluation on up to 512 NVIDIA A100 GPUs of the Perlmutter supercomputer shows that *SectorQ* reduces the simulation time by 4.4-9.7× on 16 GPUs and increases the largest finished problem by 140× on 512 GPUs compared with *cuPauliProp*, the existing multi-GPU engine. We open-source *SectorQ* at: https://anonymous.4open.science/r/SectorQ

2 Background

2.1 Pauli Propagation beyond 100 Qubits

Pauli propagation represents an observable as a sparse sum of Pauli strings and applies circuit operations in the Heisenberg picture [3, 17]. The propagated observable is evaluated against the initial state to obtain an expectation value. This representation has supported circuits with more than 100 qubits, with execution cost determined by the retained expansion, string length, and gate sequence.

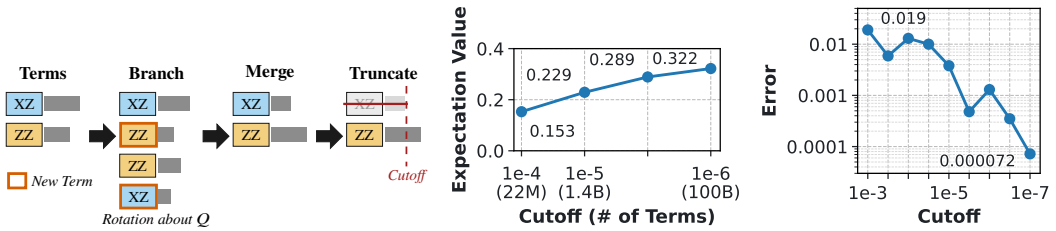
Branch, Merge, and Truncate. Each term consists of a coefficient and a Pauli string, represented by one x bit and one z bit per qubit. The product of two strings corresponds to the XOR of their bit vectors, with the multiplication phase included in the coefficient. A rotation with generator Q preserves a commuting string and branches an anticommuting string P into strings P and $P \oplus Q$ with cosine and sine factors, respectively. One rotation can consequently double the term population before merging. The merge sums coefficients with identical strings, and truncation removes terms whose merged coefficient magnitude is below the cutoff. Clifford operations map each string to one string, while the Pauli noise channels considered here scale coefficients without changing their strings (Figure 1). The expansion size changes at branching, merging, and truncation, requiring storage for intermediate terms as well as retained terms.

Term Growth with the Coefficient Cutoff. The coefficient cutoff controls the retained expansion and affects the approximation error. On a 30-qubit circuit with an exact reference, the absolute error is 0.019 at cutoff 10^{-3} and 0.00007 at 10^{-7}. On the 127-qubit kicked Ising circuit, the expectation value is 0.153 at cutoff 10^{-4}, 0.229 at 10^{-5}, and 0.322 at 10^{-6}. The largest boundary count rises to 159 million terms at cutoff 10^{-5} and 11.9 billion at 10^{-6}. The latter occupies about 475 GB at 40 bytes per term before execution buffers. Within a window, we also sum the maximum term count reached by each sector execution. The largest of these sums reaches 1.4 billion and 100 billion at the two cutoffs, respectively. These sums quantify intermediate expansion across sector executions, whose maxima can occur at different times. The boundary population measures stored operator size, while the sector maxima characterize the additional working-buffer requirement.

Merge before Truncation. Coefficient truncation applies to the sum of all contributions with the same Pauli string. Contributions from different input terms can individually fall below the cutoff even when their sum exceeds it. Independent truncation of those contributions changes the retained operator. For example, the 127-qubit kicked Ising circuit at cutoff 10^{-4} produces expectation values

Table 2. Distribution of state vector simulation and Pauli propagation

	State Vector	Pauli Propagation
Representation	2^n amplitudes	Variable number of Pauli terms
Index space	Fixed amplitude indices	Strings retained during propagation
Gate operation	Updates indexed amplitudes	Branches, merges, and truncates terms
Remote dependence	Distributed amplitude indices	Equal strings across partitions
Storage requirement	2^n complex amplitudes	Retained terms and intermediate expansion



(a) Operations of one gate (b) Kicked Ising value (c) 30-qubit TFIM error

Fig. 1. Operations and accuracy of Pauli propagation

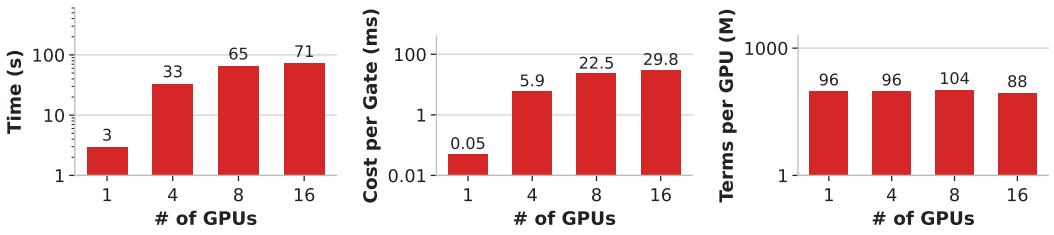
of 0.194-0.283 with truncation before the global merge, compared with 0.1531 after the merge. The gate generator determines which input strings can contribute to an identical output string. A distribution based only on the current locations of input terms cannot guarantee that these contributions are local. A locality-preserving partition must constrain generator-induced changes to string labels while accommodating the expansion's storage requirements (Table 2).

2.2 Challenges in Distributed Pauli Propagation

Distributed propagation must combine contributions across partitions before truncation and provide sufficient memory for intermediate expansion. We examine these requirements in *cuPauliProp* v0.5.0 using the 127-qubit kicked Ising circuit on NVIDIA A100 GPUs.

Global Step after Every Gate. *cuPauliProp* performs a distributed merge after every gate because contributions to an identical string may reside on different GPUs. Each MPI rank applies the gate to its local terms and issues 22 allgather calls during merge and redistribution in the measured trace. These collectives occur even though 57% of the gates exchange no terms. This global step adds a fixed cost of 5.9, 22.5, and 29.8 ms per gate on 4, 8, and 16 GPUs, against 0.05-0.8 ms on one GPU (Figure 2). The measured workload reaches 22.5 million intermediate terms and takes 2.9-3.2 s on one GPU, compared with 33-48, 65, and 71 s on 4, 8, and 16 GPUs. GPU computation occupies 9% of execution time in this trace. The largest rank holds 1.001 times the mean term count, indicating nearly uniform term distribution. The increasing gate-level coordination cost limits performance despite this balance.

Capacity Bound by the GPU Memory. *cuPauliProp* stores each rank's expansion and execution workspace in GPU memory. Across the seven circuits, intermediate term counts reach 2.9-36 times the population at the start of a gate group. Merge workspace requires additional memory alongside these intermediate terms. *cuPauliProp* exhausts memory at 88-104 million retained terms per rank on 1-16 GPUs, although the retained payload occupies only about 4 GB of the 36 GB device-memory budget. Deferred deduplication reduces merge frequency, but duplicate strings exhaust the memory



(a) Run time (b) Fixed cost per gate (c) Terms per GPU at failure

Fig. 2. Distributed mode of *cuPauliProp*

of 16 GPUs in two of the seven circuits. Without the global merge, local truncation also changes the expectation value to 0.1675 on four GPUs and 0.189 on 16 GPUs, compared with the merged value of 0.1531. Available storage must accommodate intermediate expansion and workspace while preserving the merge required for coefficient truncation.

2.3 Opportunity in Parities Conserved by Windows

A partition conserved across several gates can permit local merge and truncation throughout those gates. We examine the available conserved parities in seven circuits and their distribution over kicked Ising terms.

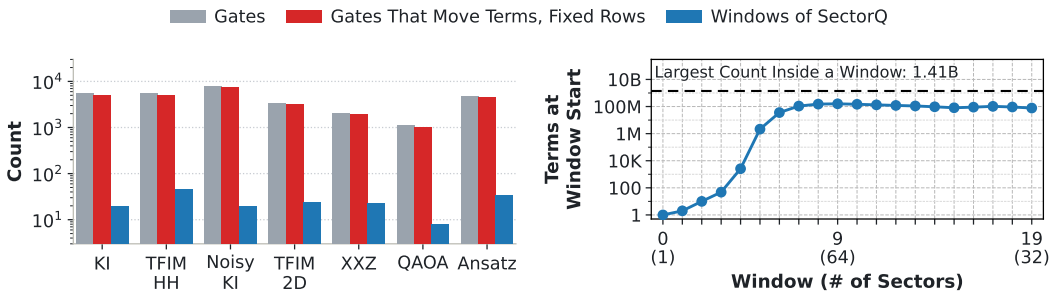
Parities Conserved by Windows of Gates. A rotation with generator Q can produce the string $P \oplus Q$ from P. Every row r satisfying $r \cdot Q = 0$ assigns the same parity to both strings. A window of consecutive gates conserves rows orthogonal to all its generators, permitting independent propagation within the resulting sectors. Table 3 counts windows under a rule that retains at least n independent conserved rows for an n-qubit circuit. The seven circuits contain 1,100-7,960 gates and form 8-45 windows, reducing potential term-exchange boundaries by factors of 89-398 relative to gate boundaries. The full circuits conserve at most two independent parities in this analysis, and their live terms have identical values on those parities. Full-circuit conserved rows consequently cannot divide the measured expansions, whereas window-specific rows provide nontrivial partitions.

Communication with Fixed Partitions. The linear partition in paulistrings-rs retains fixed rows throughout propagation [12]. In the following analysis, gates act directly on strings without a Clifford frame. A generator that changes a partition label requires inter-partition communication when the affected terms are present. With 16 parts, random rows over all bits permit label changes at 93-96% of the gates in the seven circuits, compared with 31-95% for rows restricted to x bits or z bits. On the kicked Ising circuit, random rows change the owner of 94% of the terms at a layer of two-qubit gates, and rows over lattice regions reduce this share to 68-89% with the largest of 16 parts at 1.65-3.37 times the mean. The implementation lists automatic row selection as an open problem [12]. A row change requires repartitioning the retained strings, making both its frequency and the resulting load balance relevant to execution cost.

Partition Selection from Live Terms. Window generators determine valid rows, while the distribution of live strings determines the population of each sector. For kicked Ising at cutoff 10^{-5}, the population at window starts increases from one to 159 million terms (Figure 3). Early windows have insufficient terms to occupy all GPUs, whereas later windows require multiple sectors per node. Intermediate growth further changes sector sizes within a window. A row set that balances one boundary population need not balance the population at another boundary, even

Table 3. Window counts and fixed-partition communication in seven circuits

Circuit	Gates	Windows	Gates per Window	Moving Gates (Random Rows)	Moving Gates (z Rows)
Kicked Ising	5,420	20	271	94%	50%
TFIM heavy-hex	5,420	45	120	93%	50%
Noisy kicked Ising	7,960	20	398	95%	50%
TFIM 11×11	3,410	24	142	94%	60%
XXZ 8×8	2,048	23	89	94%	62%
QAOA	1,100	8	138	94%	60%
Ansatz	4,784	34	141	96%	95%



(a) Gates and windows

(b) Terms at window starts

Fig. 3. Windows of gates in seven circuits

when it remains algebraically valid. The system can use the live distribution to select rows and sector counts subject to the conservation constraints. This separation between validity and load balance permits a partition that preserves merge locality while adapting execution granularity to the retained operator.

The design in Section 3 combines window-conserved partitions with run-time row selection and host-memory sector storage. The partition supplies merge locality, while host storage accommodates sectors beyond the working set of an individual GPU.

3 SectorQ Design

SectorQ addresses the locality and capacity limits with three mechanisms. Window-conserved parity sectors assign equal-string contributions to the same sector throughout a group of gates. Runtime re-sectoring selects the next partition from window generators and sampled live terms during execution. Host-memory sector execution separates the stored expansion from the working set of each GPU and redistributes terms at window boundaries.

3.1 Overall Architecture

Figure 4 shows the circuit processing and the execution loop on each node. The CPU constructs windows and selects their rows, host memory holds the sectors, and each GPU processes one sector at a time.

Window Formation. The *Window Builder* first expresses the circuit in the processing order of the observable. A Clifford frame absorbs eligible Clifford gates and transforms the generators of subsequent rotations, preserving the action on the observable. For example, a Hadamard maps the

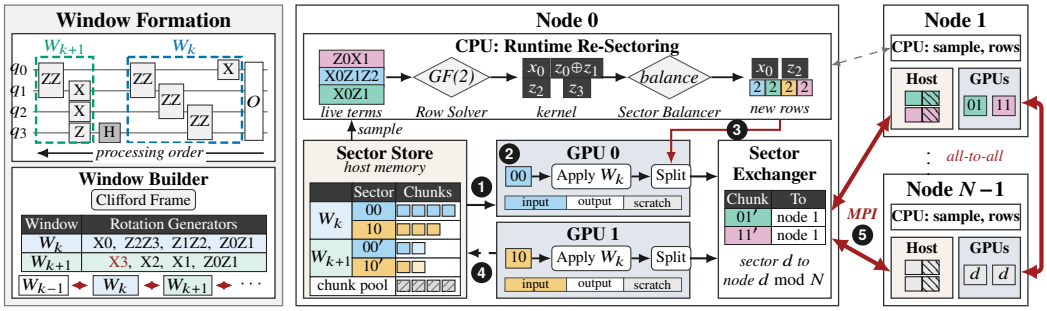


Fig. 4. Overall architecture of *SectorQ*

generator Z_3 to X_3 in the illustrated circuit. For each window, the builder accumulates the span of the branching generators over GF(2) and stops before its dimension exceeds the number of qubits. This bound ensures at least as many independent conserved rows as qubits for run-time partition selection. The default boundary moves to the last scheduled truncation in the second half of the window when one is available, reducing the terms that require redistribution. Clifford operations that remain explicit transform generators into a common coordinate system for this calculation. The resulting window defines a sequence of operations and a space of valid partitions, with the particular rows selected during execution.

Window Loop. Each node follows the numbered steps in Figure 4. At step (1), a GPU worker loads one sector from the *Sector Store* in host memory. At step (2), its *Sector Executor* applies the current window, including the merges and scheduled truncations. During these operations, *Runtime Re-Sectoring* uses the CPU to select the rows of the next window. At step (3), the worker uses these rows to partition its output on the GPU. Step (4) appends output chunks to the host lists of the next sectors. The *Sector Exchanger* transfers the remote lists at step (5), assigning sector d to node d mod N among N nodes. Each node then processes its received sectors with the same loop. The last window evaluates the observable against the input state and reduces the partial answers across nodes. Each sector is complete before its next window begins, permitting the window's merges and truncations to execute locally.

3.2 Window-Conserved Parity Sectors

The execution loop requires a partition that preserves merge locality through every operation of a window. Figure 5 illustrates this condition on four qubits and two parity rows.

Rows Conserved by a Window. A Pauli string has a binary representation $v = (x, z) \in \mathrm{GF}(2)^{2n}$, with its phase represented in the coefficient. For a row r, the parity is the binary inner product $r \cdot v$. A rotation with generator q branches an anticommuting input string v into v and $v \oplus q$. Their parities agree whenever $r \cdot q = 0$. The *Row Solver* computes the annihilator of the generator span with Gaussian elimination over GF(2). If that span has dimension d, its annihilator has dimension $2n - d$, at least n under the window rule. The selected rows form a matrix R, and the bit vector Rv labels the sector of v. This binary linear algebra is related to symmetry detection for qubit tapering [4]. The rows label terms of the observable during one window, retaining the full qubit representation for subsequent windows.

Merge Locality. Every descendant of a term retains its sector label through the branching rotations of the window. This property follows by induction from $R(v \oplus q) = Rv$ for each generator in the window. A coefficient change from a Pauli noise channel also preserves the string and its label. For an explicit Clifford operation, the row coordinates transform with the strings to preserve

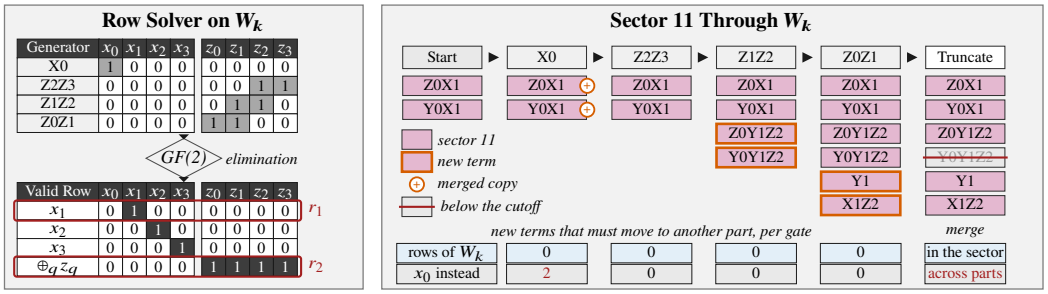


Fig. 5. Window-conserved parity sectors

the same label. Equal strings have equal labels, placing all contributions to a string in one sector before truncation. Consequently, a merge and coefficient truncation within each complete sector produce the same operator as a global merge and truncation at the same operations, apart from floating-point summation order. In the example, the rows x_1 and $z_0 \oplus z_1 \oplus z_2 \oplus z_3$ label both Z_0X_1 and Y_0X_1 as sector 11. The descendants retain label 11 throughout W_k, allowing their coefficients to be combined before the scheduled truncation.

Exchange Only at Window Boundaries. The next window can contain generators that change the current parity labels. For example, X_1 in W_{k+1} changes the parity x_1 used by W_k. The runtime computes a new matrix for W_{k+1} and regroups the output of W_k before that window begins. This regrouping collects all fragments of each next sector on its owner for merging before the next truncation. The construction differs from a fixed linear partition [12], whose rows can require a remote transfer at a gate that changes a partition label. It also differs from a partition based only on symmetries of the full circuit, since these symmetries need not distinguish the live terms. The window rule requires conservation only over the operations assigned to the current sector execution. The resulting partitions can change at window boundaries while preserving the prescribed truncation schedule.

3.3 Runtime Re-Sectoring

Conservation determines which rows are valid, but the live terms determine their load balance. Figure 6 shows how the runtime selects the next rows while the GPUs process the current window.

Sampling the Live Terms. At the start of a window, each node samples strings from its host sectors with a regular stride determined by the global term count. The configured sample target is 200,000 terms across all nodes. The CPU applies the current window's bit-changing Clifford operations to these strings before the next partition is scored. This estimate applies coordinate changes without reproducing the branching and truncation of the sampled operator. The sample approximates the string coordinates at the next boundary, with no exact prediction of the output population. An MPI gather collects the samples on rank zero, where the row solver and the *Sector Balancer* make a common decision for every node. A broadcast distributes the selected rows and window description. All workers use the broadcast matrix to assign identical labels to identical strings. The sample affects performance decisions only, since every selected row must satisfy the generator constraints independently of the sampled terms.

Balanced Row Selection. The sector balancer generates eight candidate row sets from random combinations of the annihilator basis. For each candidate, it counts sampled strings per sector and selects the row set that minimizes the maximum count. The illustrated pairs produce largest counts of eight, four, and two, and the selected pair (x_0, z_2) divides the eight strings equally. The number

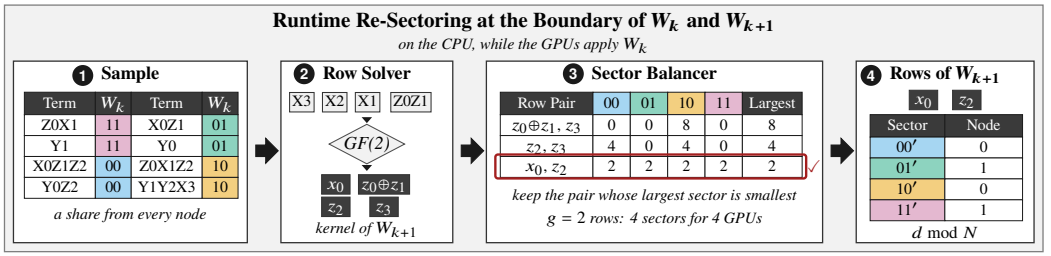


Fig. 6. Runtime re-sectoring at a window boundary

of rows depends on the estimated output size, the target terms per sector, and the available GPUs. The estimate uses the previous window's observed growth with a bounded growth factor. The runtime requests one task per GPU, or two when the preceding window exceeds the configured duration threshold, provided the resulting sectors remain sufficiently large. Windows with few terms use fewer sectors to avoid small GPU tasks. A configurable cap bounds the number of sectors at a boundary. Candidate scoring controls the sampled maximum sector size, while the row count controls execution granularity and the number of available tasks.

Overlap with the Running Window. The host sample is captured before the workers consume their input sectors. After the workers start, the CPU main thread gathers the sample, computes the next window, scores the candidate rows, and broadcasts the result. Workers need those rows only when they partition their output for the next window. A worker that reaches this point before the rows are ready waits on a local condition variable. The implementation records this wait separately from CPU selection time to distinguish overlapped selection from a delay visible to workers. The overlap depends on the relative durations of row selection and sector execution. Short windows can expose selection latency, whereas longer windows provide more time for the CPU calculation before output partitioning. The conservation constraints ensure merge locality in both cases, independent of sample quality and the duration of selection.

3.4 Host-Memory Sector Execution and Exchange

The selected sectors are units of execution rather than permanent GPU allocations. Figure 7 shows their storage, processing, and transfer across a window boundary.

Chunked Sector Store. The sector store represents each sector as a list of fixed-capacity host chunks, with 262,144 terms per chunk by default. Each term contains a binary string and a double-precision coefficient. A pool retains pinned chunks for reuse after GPU transfers or MPI sends, avoiding repeated registration for every sector. The same chunk representation supports input loading, output collection, and reception from another node. A sector can contain multiple chunks, and a node can hold more sectors than it has GPUs. The live expansion occupies host memory, while only the sectors under execution require GPU buffers. The total capacity depends on host storage for input, temporary output, and exchanged chunks. Within a node, workers obtain sectors from a shared queue, initially ordered by decreasing size. Workers obtain another sector as they finish their current task, redistributing work within the node as sector execution times vary.

Sector Execution on GPUs. Each worker maintains input and output expansions together with the library workspace on its GPU. The worker loads a complete sector, applies its window, and performs the required merges before coefficient truncation. If the sector is too large to load with sufficient workspace, an additional valid row divides it in host memory. If growth exhausts the GPU buffers during execution, the worker partitions the current expansion using a further row

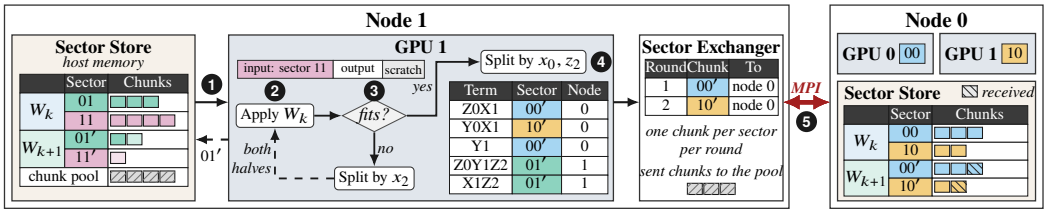


Fig. 7. Host-memory sector execution and boundary exchange with pale filling chunks, black-hatched received chunks, and gray-hatched pooled chunks

conserved by the remaining operations. The resulting subsectors return to the local queue with the operation position and any pending merge or truncation. Each subsector resumes from that position and preserves merge locality under the finer partition. A split succeeds only if the available rows distinguish enough terms to fit the working set. The runtime records host and GPU splits separately. Host storage can accommodate a larger operator only when its sectors admit subdivisions whose expansions and workspace fit an individual GPU.

Boundary Exchange. At the end of a window, a GPU partitions its surviving terms by the next matrix and copies them into host chunks. Chunks for local sectors remain on the node, and the sector exchanger sends the other chunks to their owners. The exchange first communicates counts and then transfers the chunks in rounds. For each sender and destination sector, a round transfers at most one chunk. After the outstanding requests complete, the sender returns transmitted chunks to the pool before the next round begins. Additional receive storage is bounded by one round, and the request count depends on active sender-sector pairs. At completion, the owner joins the received chunk lists into each complete next sector. A checkpoint can record this boundary state and its next-window description. Chunk reuse permits output collection and redistribution without allocating a second receive buffer for the entire expansion.

3.5 Implementation

SectorQ implements the host runtime in C++ and the string partition operations in CUDA. Each MPI rank owns one node and creates one worker per GPU, with four GPUs per node in our experiments.

Runtime and Library Operations. Each worker uses *cuPauliProp* v0.5.0 on one GPU. The library operations cover rotations, Clifford gates, Pauli noise, merging, and truncation. The runtime supplies the sector contents and manages their placement, while the library supplies the arithmetic on each expansion. CUDA kernels evaluate the row parities and group output terms by their next sectors. The CPU handles binary elimination, sample scoring, the host chunk pool, and MPI exchange. Consecutive Pauli noise channels can share a library call up to the next truncation, preserving the scheduled coefficient decisions. On one GPU, an expansion below the target sector size can remain in device memory between windows. This path avoids a host transfer when no other worker could receive a sector. Both optimizations preserve the window partition and its scheduled coefficient decisions. Table 4 summarizes the default parameters and the larger sector caps used in the capacity experiments.

4 Evaluation

4.1 Evaluation Setup

We evaluate execution time, scaling and capacity, design contributions, numerical agreement, and parameter sensitivity under the following conditions.

Table 4. Runtime parameters

Parameter	Setting
Sample target across nodes	200,000 terms
Candidate row sets	8
Target sector input	4 million terms, 6 million for kicked Ising
Tasks per GPU	1, or 2 after a window of at least 0.5 s
Default sector cap	256
Capacity experiment sector caps	1,024 or 4,096
Host chunk capacity	262,144 terms, 65,536 for selected large runs
GPU memory budget	90% of available device memory
Each expansion buffer	20% of the GPU memory budget
Coefficient truncation	Every 10 operations by default

Hardware Specification. All experiments use NERSC Perlmutter GPU nodes, each with four NVIDIA A100 GPUs with 40 GB of device memory each, a 64-core CPU, and 256 GiB of host memory. The nodes communicate over the Slingshot network. GPU implementations use 90% of available device memory, approximately 36 GB per GPU. *SectorQ* assigns one MPI rank to each node and one worker to each GPU, whereas distributed *cuPauliProp* assigns one rank to each GPU. The largest configuration contains 128 nodes and 512 GPUs. CPU implementations use the same nodes as the corresponding GPU configuration. ORQA uses 64 ranks per node, MonoProp uses eight ranks with eight threads each, and threaded single-node implementations use 64 threads. Qiskit uses one thread. CPU and GPU comparisons use equal node counts, with the rank and thread configurations specified above.

Evaluation Scope. The experiments measure Heisenberg-picture propagation that returns an observable expectation value. Unless stated otherwise, each configuration is measured once with a 40-minute limit. The size and exact-reference experiments use a 20-minute limit, and capacity ladders allow *SectorQ* up to two hours per run. A cutoff sweep stops after memory exhaustion or its time limit. The plots distinguish completed measurements, observed failures, and unexecuted configurations. Reported times cover propagation and any initialization performed lazily within the timed call, excluding queue and process-launch delays. Horizontal-axis term counts denote the largest population at a *SectorQ* window boundary under the stated truncation rule. These counts differ from intermediate expansion within a window and the final output size. Separate exact-reference and repeated-run experiments assess approximation error and timing variability.

Circuits. The seven circuits in Table 3 vary the interaction graph, gate family, and observable. The principal workload is a 127-qubit heavy-hex kicked Ising circuit with 20 steps and rotation angle $\pi/4$. A second heavy-hex circuit uses transverse-field Ising model (TFIM) rotations at general angles. Noisy kicked Ising adds depolarizing channels with probability 0.005 to the principal workload. The remaining circuits are TFIM on an 11×11 lattice, XXZ on an 8×8 lattice, a 100-qubit QAOA circuit on a random regular graph, and a 100-qubit hardware-efficient ansatz. Their respective depths are 10 steps, 6 steps, 4 layers, and 16 layers. The observables include local magnetization and the edge sum of QAOA. Additional TFIM grids and kicked Ising depths vary circuit dimensions. Every comparison within a setting preserves the circuit, observable, and initial state.

Baselines. The primary baseline is *cuPauliProp* v0.5.0 with the original circuit and its default distributed mode. The second baseline uses a Clifford-frame input and defers global deduplication until truncation, with additional merges where required to fit memory. We report the fastest successful deferred interval among those tested. The paulistrings-rs comparison uses original gates and

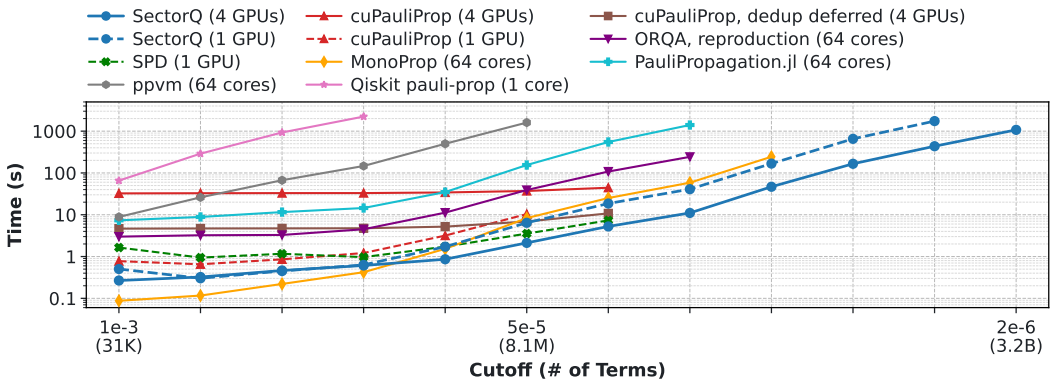


Fig. 8. Simulation time on one node (4 GPUs)

truncation after each operation in both systems to match its gate-level coefficient decisions. Other comparisons use the authors' SPD, MonoProp, PauliPropagation.jl, and ppvm implementations, Qiskit's Pauli propagation package, and our reproduction of ORQA. The external implementations other than paulistrings-rs receive the exported Clifford-frame operation list. MonoProp has a different truncation rule and requires separate error measurements for a comparison at equal accuracy. The ORQA results describe our reproduction rather than the authors' implementation. The default *SectorQ* truncation interval is ten operations, with gate-level truncation used for the matched paulistrings-rs comparison.

4.2 Performance Compared with Existing Systems

The comparisons cover one node, seven circuits on 16 GPUs, and three circuits on 128 GPUs. The paulistrings-rs experiment uses a separate matched truncation schedule.

Single-Node Performance. Figure 8 compares propagation time and completed cutoff ranges on one node. Across circuits that fit one GPU, the ratio of the *cuPauliProp* time to the *SectorQ* time is 0.6-1.5. SPD is up to 2.9 times faster on one GPU, but its kicked Ising run exhausts memory at cutoff 10^{-5}. At their shared cutoff 3×10^{-5}, *SectorQ* on four GPUs takes 5.25 s and distributed *cuPauliProp* takes 44.3 s. *SectorQ* also completes 3×10^{-6} in 436 s. At the largest shared settings across circuits, the four-GPU implementation is 108-269 times faster than PauliPropagation.jl, 157-755 than ppvm, and over 1,800 than single-threaded Qiskit. The ORQA reproduction is 18-71 times slower on the same node. MonoProp remains faster on several small settings below two seconds, including kicked Ising at 10^{-3}. The small-expansion measurements provide no consistent advantage for *SectorQ*. Its four-GPU execution completes larger expansions after several single-device implementations exhaust memory.

Seven Circuits on 16 GPUs. Figure 9 compares the seven circuits on four nodes. At the largest setting completed by the original-circuit *cuPauliProp* baseline for each circuit, *SectorQ* is 4.4-9.7 times faster. Deferred deduplication remains 2.2-4.6 times slower at the shared settings and exhausts memory in two circuits at the primary baseline's largest completed setting. *SectorQ* completes lower cutoffs in all seven circuits, including TFIM and XXZ inputs without removable Clifford gates. The TFIM and XXZ results demonstrate a reduction in propagation time even without removable Clifford operations. For context, total propagation time divided by the maximum boundary count is 17-130 ns per term for *SectorQ* above 100 million terms. The primary baseline records 908-39,000 ns

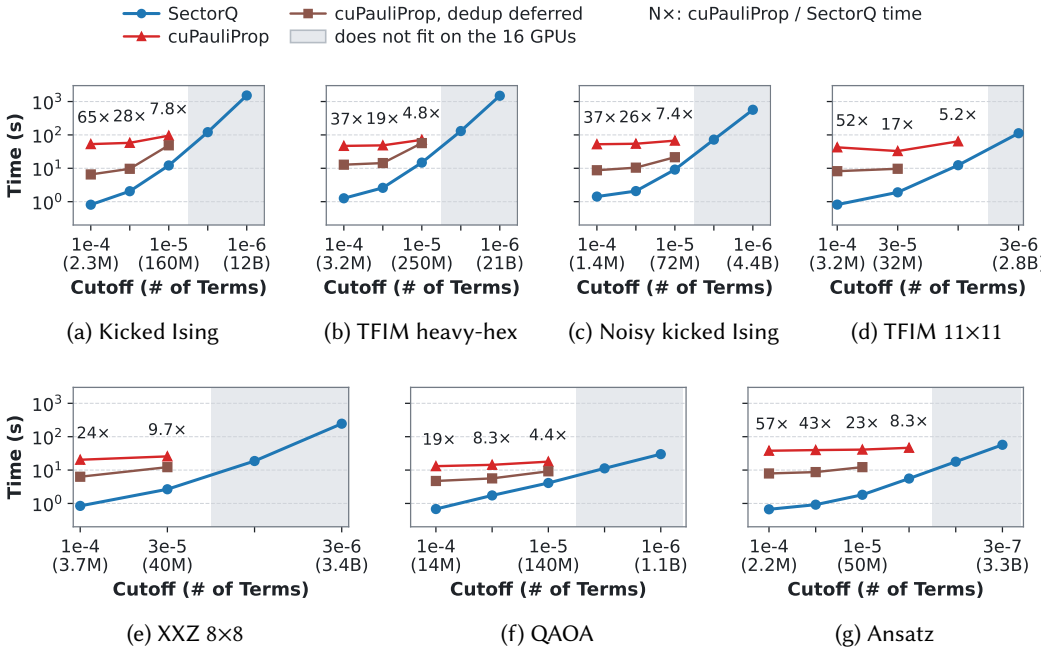
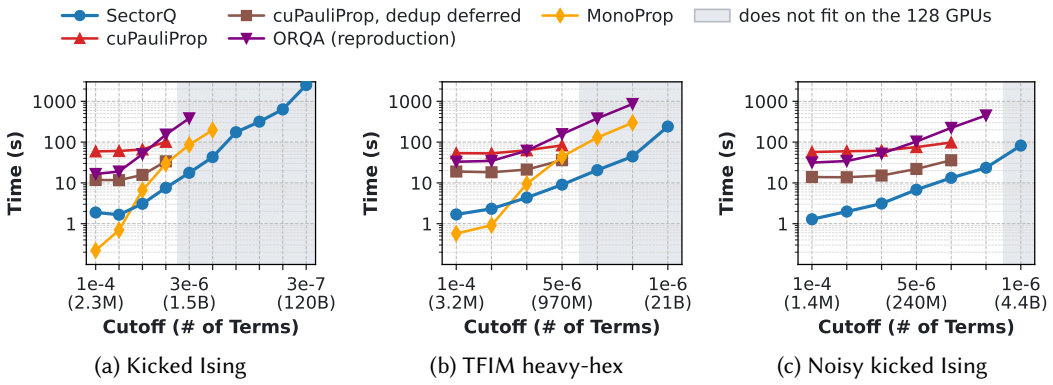


Fig. 9. Simulation time of seven circuits on 16 GPUs

per term at cutoff 10^{-4}. These values describe amortized propagation time at different operator sizes, not a matched-size speedup or the cost of one gate per term.

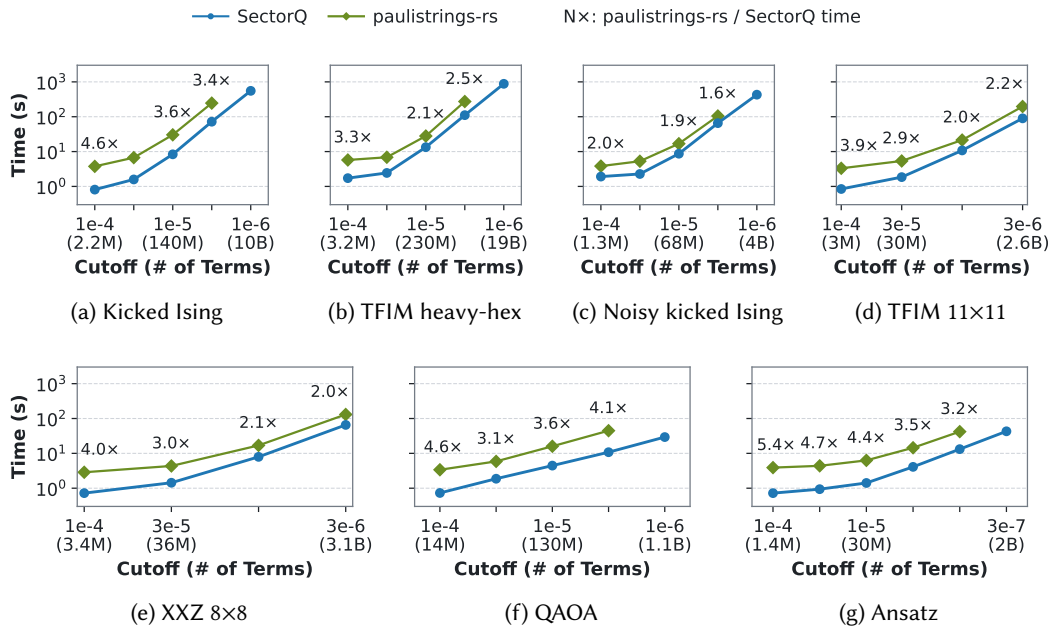
Three Circuits on 128 GPUs. Figure 10 compares three circuits on 32 nodes using the original-circuit *cuPauliProp* baseline. Kicked Ising and its noisy variant use those original inputs, while the TFIM input is identical with and without a Clifford frame. At the largest setting completed by the primary baseline in each sweep, *SectorQ* is 7.4-13.4 times faster. For kicked Ising at cutoff 5×10^{-6}, the times are about 7.6 and 101.6 s. The CPU comparisons use the same 32 nodes. MonoProp remains competitive at the smallest settings, subject to its different truncation rule. At the largest shared cutoffs completed by the ORQA reproduction, *SectorQ* is 19.2-21.7 times faster. These cutoffs are 3×10^{-6} for kicked Ising and 2×10^{-6} for TFIM and noisy kicked Ising. The next ORQA setting exhausts memory for kicked Ising and its noisy variant, while the next TFIM setting is excluded by the time estimate.

Fixed-Partition Implementation with Matched Truncation. Figure 11 compares *SectorQ* with paulistrings-rs on 16 GPUs using truncation after each operation and the same original circuit. The fixed partition uses the library's seeded rows. *SectorQ* is 1.6-5.4 times faster over all 30 shared settings and 1.6-4.1 times faster at the largest shared setting of each circuit. At the next tested cutoff in five circuits, paulistrings-rs exhausts GPU memory while *SectorQ* completes in 29-880 s. Times at small configurations include any lazy GPU kernel compilation within the propagation call. This comparison includes differences in arithmetic kernels, storage, and partition management under a common truncation schedule. The same-engine ablation in Section 4.4 isolates row recomputation, while Section 4.5 verifies the numerical agreement of the matched schedules.



(a) Kicked Ising (b) TFIM heavy-hex (c) Noisy kicked Ising

Fig. 10. Simulation time of three circuits on 128 GPUs



(a) Kicked Ising (b) TFIM heavy-hex (c) Noisy kicked Ising (d) TFIM 11×11

(e) XXZ 8×8 (f) QAOA (g) Ansatz

Fig. 11. *SectorQ* and fixed linear partitions on 16 GPUs

4.3 Scalability and Capacity

We vary the GPU count, circuit dimensions, and retained operator size to distinguish execution scaling from capacity.

Strong Scalability. For fixed large problems, *SectorQ* achieves speedups of 81-166 on 256 GPUs and 83-180 on 512 GPUs relative to one GPU. The representative settings are kicked Ising at cutoff 3×10^{-6} and heavy-hex TFIM and noisy kicked Ising at 5×10^{-6}. Figure 12 also shows the kicked Ising cutoff ladder from one to 512 GPUs. From 32 GPUs onward, its capacity measurements at cutoffs 10^{-6} and below use higher sector caps and, on 128 GPUs onward, smaller chunks. The default cap of 256 sectors limits additional speedup beyond 256 GPUs. At kicked Ising cutoff 3×10^{-6}, time decreases from 10.48 to 9.67 s. The trace contains at most 256 nonempty sectors and no GPU

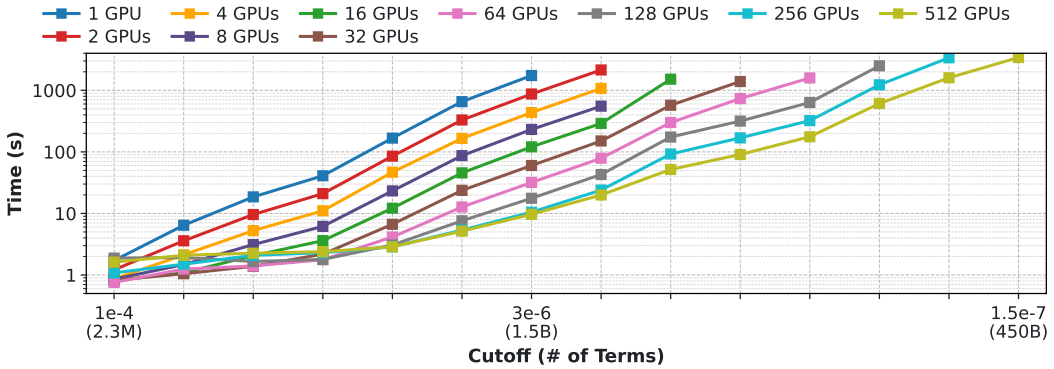


Fig. 12. Kicked Ising propagation from 1 to 512 GPUs

splits, leaving fewer concurrent tasks than the 512 available workers. On 512 GPUs, TFIM at cutoff 2×10^{-6} takes 20.7 s with *SectorQ* and 201.2 s with original-circuit *cuPauliProp*. The next *cuPauliProp* setting, 10^{-6}, exhausts GPU memory.

Weak Scalability. Figure 13 increases the boundary term count with the GPU count by lowering the cutoff. Efficiency is boundary-term throughput per GPU relative to the corresponding one-GPU run, accounting for the counts actually produced. For the large load, *SectorQ* retains 80-88% of this throughput on 16 GPUs and 72-80% on 64 GPUs across the three circuits. The range falls to 23-38% on 512 GPUs under the default sector cap. The measured large-load *cuPauliProp* configurations exhaust GPU memory, whereas the small load permits time comparisons between both systems. On 16 GPUs, *SectorQ* is 8.1-12 times faster than the original-circuit baseline for the small load. On the same 64 nodes as the 256-GPU configuration, the ORQA reproduction takes 15.8-19.1 times longer for the small load. MonoProp takes 3.6 and 5.0 times longer for kicked Ising and TFIM, respectively, under its different truncation rule. The 512-GPU CPU and weak-load baseline configurations remain unexecuted. The retained population increases with GPU count, but throughput per GPU decreases at the largest configurations under the default sector cap.

Circuit Size and Depth. Figure 14 varies the TFIM lattice size and kicked Ising depth on 16 GPUs. The lattice sweep covers 36-1,024 qubits with total magnetization as the observable, changing string length, graph size, and observable size together. At 1,024 qubits on a 32×32 lattice, *SectorQ* completes in 90.4 s and is 4.1 times faster than original-circuit *cuPauliProp*. The sweep ends at the library's supported limit of 1,024 qubits. The depth sweep keeps the kicked Ising graph fixed and increases its Trotter steps. At 60 steps, *SectorQ* takes 160 s and is 50 times faster than the primary baseline. The lower execution time persists across changes in string length, interaction graph, and propagation depth. The term counts on the horizontal axes also vary across these configurations because retained operator size depends on the resulting branching and truncation.

Largest Operator and Host Memory. The kicked Ising capacity sweep reaches 448.2 billion boundary terms on 512 GPUs at cutoff 1.5×10^{-7}, completing in 3,413 s. The 17.9 TB expansion is 140 times larger than the largest completed original-circuit *cuPauliProp* setting on the same GPUs, 3.2 billion terms at 2×10^{-6}. The host chunk pool occupies 21,079 GiB across 128 nodes, 64% of installed host memory. This allocation includes reusable chunks and transient storage and excludes memory outside the chunk pool. On 128 GPUs, *SectorQ* completes 117.5 billion terms. On 256 GPUs, the completed endpoint is 257.3 billion terms at cutoff 2×10^{-7} in 3,382 s. Its host chunk pool occupies 11,490 GiB across 64 nodes, or 70% of installed host memory. Both CPU baselines

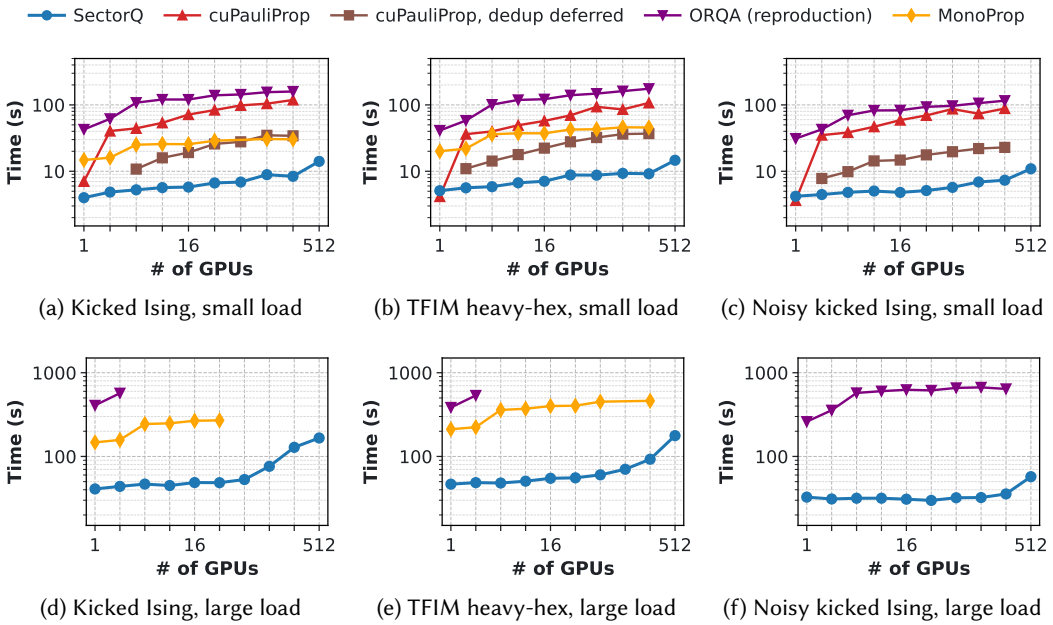


Fig. 13. Weak scaling from 1 to 512 GPUs

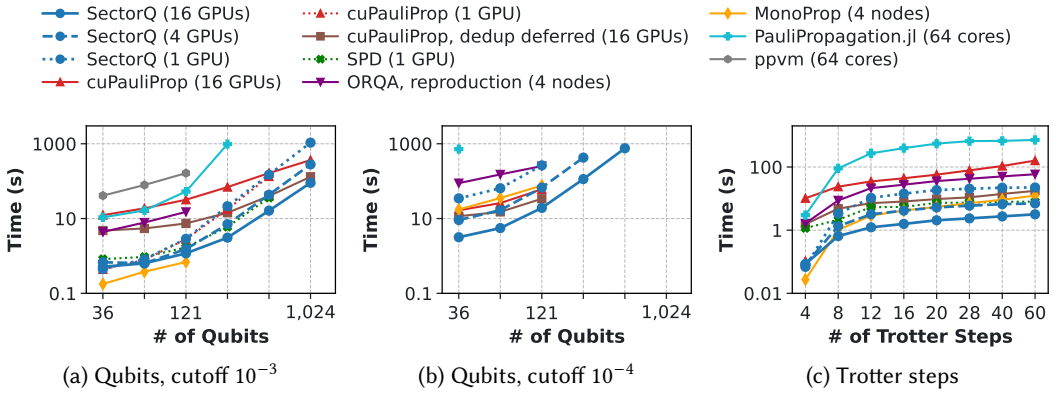


Fig. 14. Simulation time by circuit size and depth

on those 64 nodes complete kicked Ising at 2×10^{-6} and exhaust memory at 10^{-6}. Configurations rejected from estimated storage requirements remain distinct from measured failures.

4.4 Contribution of the Design Choices

Controlled engine variants evaluate preprocessing and window-dependent rows. Time counters quantify computation, copies, exchange, and worker idle time across GPU counts.

Preprocessing and Window Sectors. Figure 15 measures successive changes on kicked Ising at cutoff 10^{-5} and 16 GPUs. The original-circuit *cuPauliProp* baseline takes 95.0 s. A Clifford-frame input reduces time to 54.3 s, and deferred global deduplication reduces it to 48.8 s. Window-sector execution reaches 12.2 s with the same underlying library operations. The reduction from 48.8 to

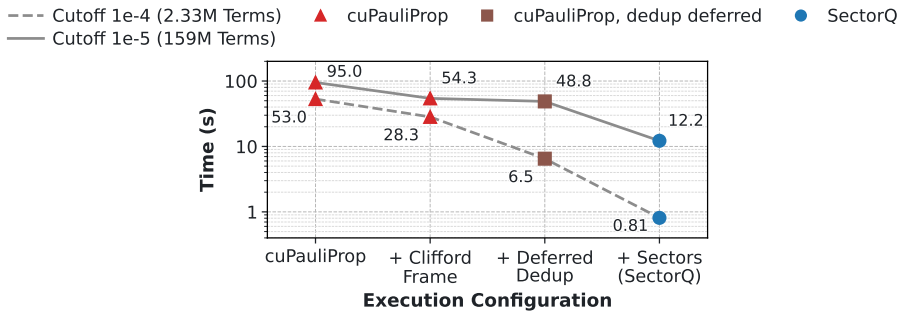


Fig. 15. Effect of preprocessing and sector execution on 16 GPUs

Table 5. Windows and inter-node traffic at cutoff 10^{-5} on 16 GPUs

	Windows		Traffic (GB)	
Circuit	**Recomputed**	**Fixed**	**Recomputed**	**Fixed**
Kicked Ising	20	878	45.2	698.5
TFIM heavy-hex	45	2,241	79.0	342.2
TFIM 11×11	24	1,518	60.0	625.9
XXZ 8×8	23	1,155	44.8	2,369.9
QAOA	8	198	8.2	34.0
Ansatz	34	1,392	8.3	107.7

12.2 s remains after Clifford preprocessing and deferred deduplication, identifying an additional benefit from sector execution. The sequence changes the surrounding execution structure while retaining the library's single-GPU arithmetic. The following fixed-row experiment compares partition rules within that sector engine. Its recomputed variant takes 11.9 s at the same cutoff in a separate allocation. The 11.9 and 12.2 s measurements are separate executions of the same design, and their difference is not attributed to another mechanism.

Fixed versus Recomputed Rows. Figure 16 isolates row recomputation with the same Clifford frame, GPU operations, and truncation schedule. The fixed variant selects rows at the first partitioned window and retains them, ending a window when a rotation changes those rows. The recomputed variant selects rows for each window. Fixed rows are 2.4-17.0 times slower over 25 successful pairs on 16 GPUs and 1.3-14.8 times slower over 22 pairs on four GPUs. All pairs have identical printed expectation values. Several fixed-row runs exhaust host memory at lower cutoffs that the recomputed variant completes. Noisy kicked Ising is excluded because the fixed variant does not support its explicit Clifford operations between rotations. Table 5 connects the time difference to window count and traffic. For kicked Ising at 10^{-5}, recomputation reduces the window count from 878 to 20 and inter-node traffic from 698.5 to 45.2 GB. Fewer boundaries reduce the frequency of sector redistribution and the associated host-device transfers.

Balanced Selection and Overlap. Figure 18 compares candidate selection and overlap within one allocation on 16 GPUs. Panels (a) and (b) contain 30 circuit-cutoff pairs, with one measurement per variant and a fixed random seed. Imbalance is the largest sector divided by the mean nonempty sector size, maximized over windows. Eight candidates reduce this maximum in 28 pairs, with its median decreasing from 1.069 to 1.029. The median one-to-eight-candidate time ratio is 1.00. Median row-selection time increases from 0.113 to 0.459 s, limiting the overall time benefit. Panel

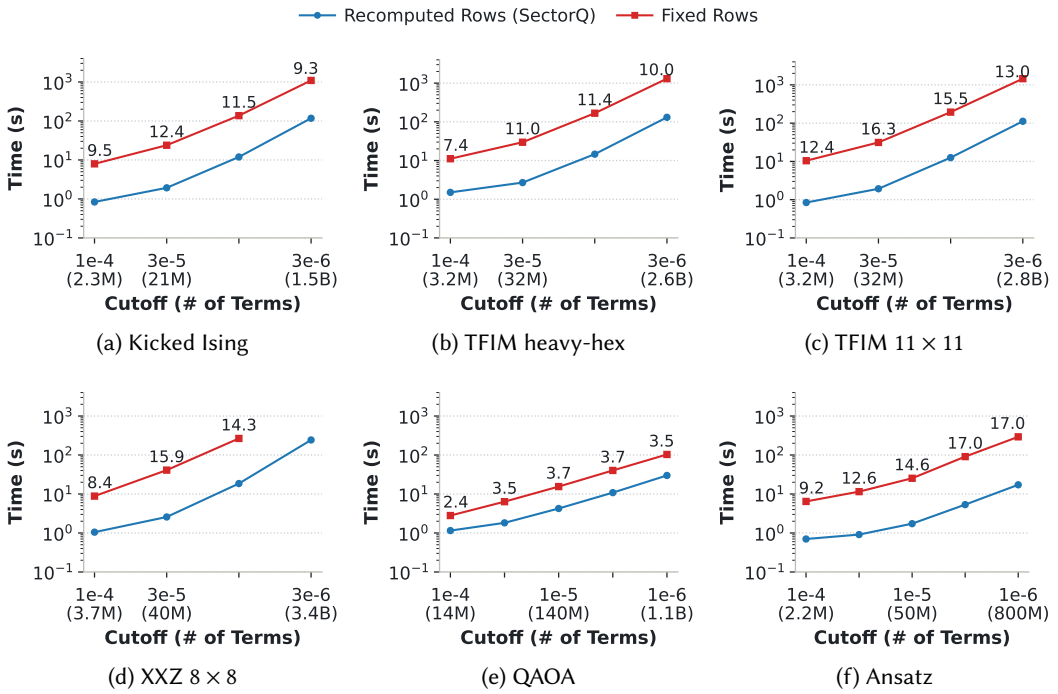


Fig. 16. Fixed and recomputed rows on 16 GPUs, with fixed-to-recomputed time ratios

(c) compares overlap enabled and disabled for three circuits and two cutoffs. Markers show ratios of mean times over three repetitions, and whiskers span the three paired ratios. Disabling overlap increases mean wall time by 3-47% across all six settings, with a geometric-mean increase of 21%. All 48 pairs print identical expectation values. Candidate competition improves balance without a consistent independent speedup, and overlap reduces the exposed selection cost.

Time Breakdown across GPU Counts. Figure 17 uses cutoff 10^{-4} as a small reference and increases the other workloads with GPU count. Within each circuit, the middle points keep boundary terms per GPU within a factor of 1.5. The 64- and 256-GPU capacity points use sector caps of 1,024 and 4,096, with 65,536-term chunks on 256 GPUs. The capacity configurations are marked with asterisks, and parentheses denote boundary term counts. Worker times are averaged over GPUs, exchange and synchronization times over nodes, and each is normalized by wall time. The Other component is the residual wall time after these recorded components. The library-call component measures elapsed call time and does not represent sampled device utilization. All 27 profiles are complete. The largest 64-GPU points contain 14.9-78.5 billion boundary terms, and the largest 256-GPU points contain 37.3-212.6 billion. Average worker idle time is below 1% at all six capacity points. The larger inputs sustain worker activity, although exchange and transfer costs remain.

Concurrency and Exchange Costs. The small reference spends 37-52% of wall time in average worker idle time across the nine profiles. At the largest 16-GPU settings, library operations occupy 76-89% of wall time and inter-node exchange occupies 3-7%. At the largest 256-GPU points, exchange occupies 10-35%, with TFIM at the upper end. The original-circuit *cuPauliProp* baseline separately records 39-44% of its large-setting time inside MPI calls. The separate 512-GPU measurements show that average worker idle time rises from 9% to 40% of wall time relative to 256 GPUs at kicked Ising cutoff 3×10^{-6}. The largest capacity run uses a higher sector cap and has a different

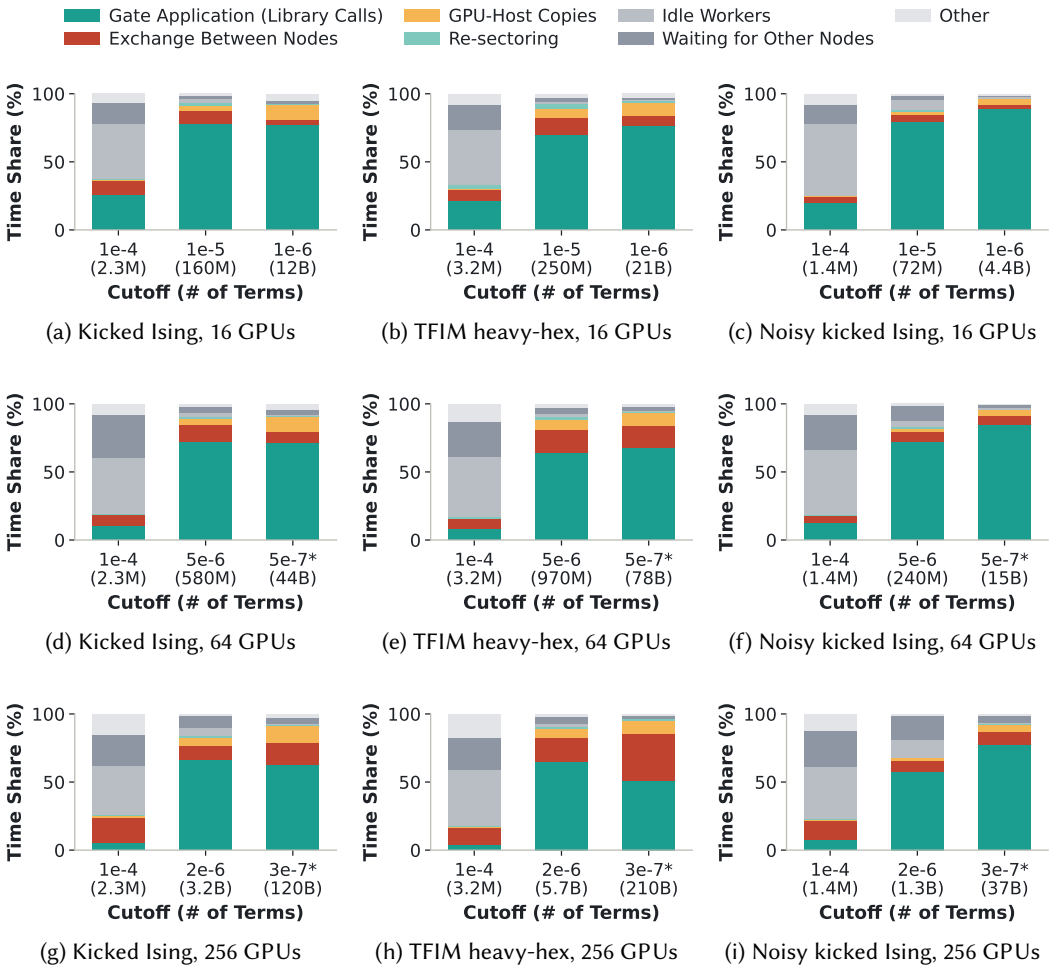


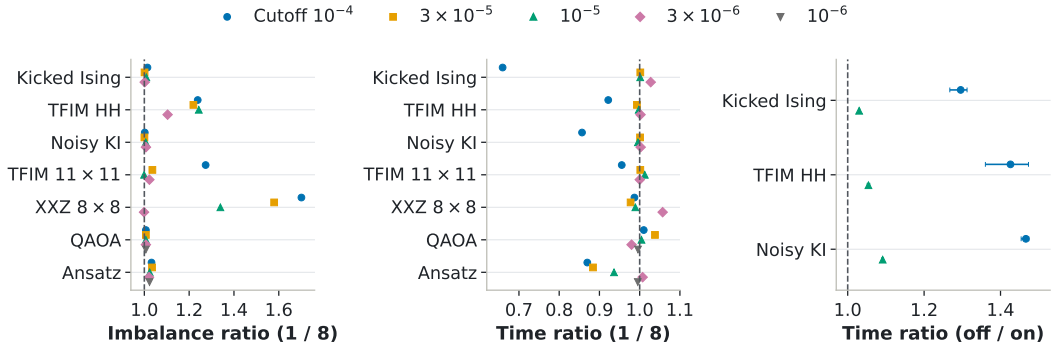
Fig. 17. Time shares for a fixed small reference and larger workloads by GPU count

composition, with 46% in library calls, 26% in exchange, and 9% in GPU-host copies. Its exchange time is concentrated in data transfer and request posting, with under two seconds spent exchanging counts. The fixed-input trace identifies limited task concurrency under the default cap, whereas the larger capacity run has a substantial exchange component despite low worker idle time.

4.5 Numerical Agreement and Accuracy

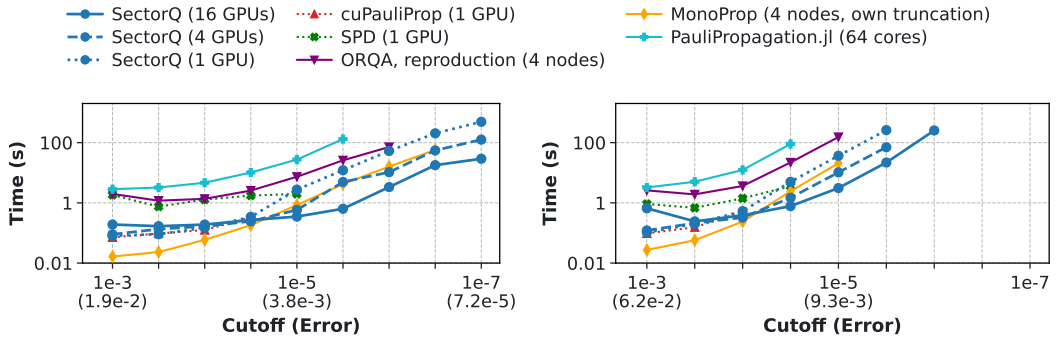
We compare implementations under matched truncation schedules and assess approximation error against an exact reference.

Agreement across Implementations. Across 68 circuit-cutoff settings with at least two successful runs under a common truncation rule, the largest absolute expectation difference is 4.9×10^{-11}. This comparison includes the single-GPU reference, distributed *cuPauliProp*, and external implementations with the same coefficient decisions. The largest difference occurs for QAOA's edge-sum observable. The separate gate-level truncation comparison with paulistrings-rs agrees within 4.7×10^{-11} over 75 completed settings on one, four, and 16 GPUs. *SectorQ* also preserves the



(a) Sector balance (b) Candidate selection time effect (c) Overlap time effect

Fig. 18. Candidate selection and overlap on 16 GPUs, with ratios above one favoring eight candidates or enabled overlap

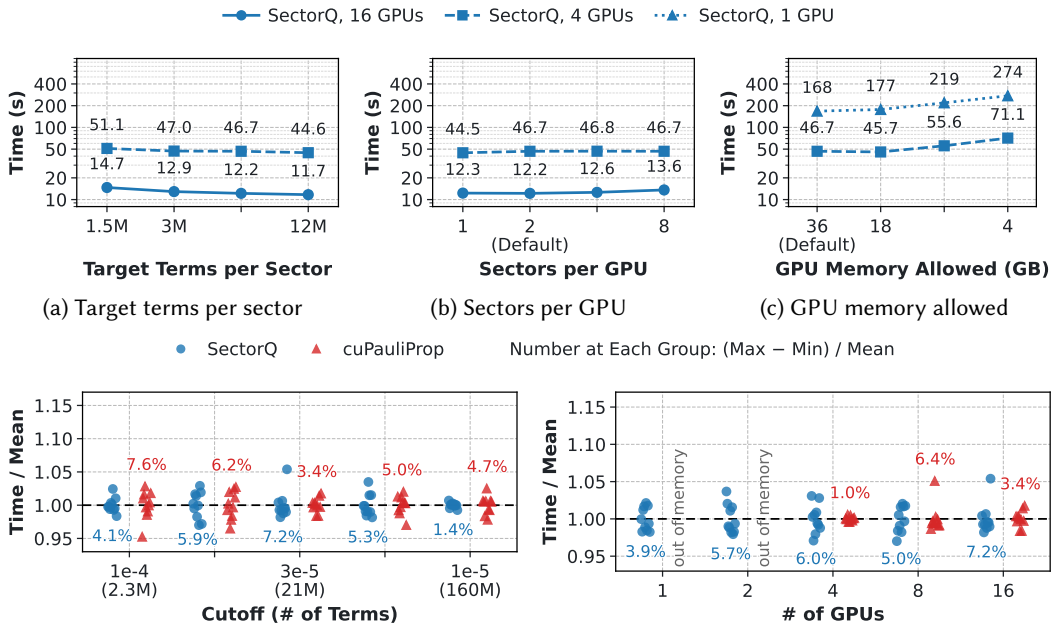


(a) 4 Trotter steps (b) 6 Trotter steps

Fig. 19. Simulation time and error of a 30-qubit TFIM

printed expectation value when sector count, GPU count, window rule, or device-memory budget changes. This agreement supports the merge-locality property across the evaluated partition and resource configurations. It measures consistency under the specified truncation schedule, while approximation error requires a reference without that truncation.

Error against an Exact Reference. Figure 19 uses a 30-qubit TFIM on a 5×6 lattice with an exact state-vector reference, checked against untruncated Pauli propagation on smaller grids. The four-step circuit reaches absolute error 7.2×10^{-5} at cutoff 10^{-7}, with *SectorQ* on 16 GPUs completing in 29.2 s. Implementations with the same truncation rule have the same error at a given cutoff, whereas MonoProp requires a separate error comparison. The six-step circuit reaches time or memory limits at higher cutoffs. Error is nonmonotonic at intermediate cutoffs in both sweeps. The large kicked Ising workload has no exact reference at the evaluated scale, and its expectation values are also nonmonotonic as the cutoff decreases. For example, the value increases from approximately 0.3217 at 10^{-6} to 0.3265 at an intermediate cutoff, then decreases to 0.3089 at 1.5×10^{-7}. The larger completed expansions establish computational capacity, but convergence of the kicked Ising expectation remains unverified.



(a) Target terms per sector (b) Sectors per GPU (c) GPU memory allowed

(d) Repetitions on 16 GPUs by problem size (e) Repetitions at cutoff 3×10^{-5} by GPU count

Fig. 20. Parameter sensitivity at cutoff 10^{-5} and ten repeated measurements of kicked Ising

4.6 Parameter Sensitivity and Repeatability

Figure 20 reports the effect of sector and memory parameters and the variability of repeated execution times.

Sector Granularity and Device Memory. At cutoff 10^{-5}, a target sector size one quarter of the default increases time by 9-26%, whereas twice the target reduces it by 2-9% across the representative circuits. One to four tasks per GPU differ by at most 9%, and eight can increase time by up to 35% on 16 GPUs. The tradeoff is between additional library invocations with small sectors and working-buffer pressure with large sectors. With a device-memory budget of 4 GB, execution time increases by 12-64% relative to 36 GB, while finer sectors preserve the tested expectation values. A larger task count can also increase total time. A 512-GPU kicked Ising run at 3×10^{-6} with a 4,096-sector cap and smaller host chunks takes 11.6 s, against the default's 9.7 s. The larger cap and smaller chunks change together in this experiment, preventing attribution of the time increase to either parameter alone.

Repeated Measurements. The repetition experiment contains 160 completed kicked Ising runs, with ten measurements per setting. We define spread as the difference between the maximum and minimum times divided by the mean of all repetitions. On 16 GPUs, spread is 1.4-7.2% for *SectorQ* and 3.4-7.6% for original-circuit *cuPauliProp*. All repetitions preserve the printed expectation value. Each system's 16-GPU repetitions use one allocation, with different allocations for the two systems. The reported spread measures variation within those allocations. At cutoff 10^{-5}, the ratio of ten-run means is 7.6, close to the main sweep's single-run ratio of 7.8. Panels (d) and (e) show the repeated times across problem sizes and GPU counts, respectively. The difference between the systems' mean times persists across the ten repetitions at this configuration.

5 Related Work

We discuss the algebraic partition methods and simulation systems relevant to window-conserved sectors, run-time row selection, and host-memory execution.

5.1 Partitions for Distributed Simulation

Prior methods identify invariant operator subspaces or group operations to reduce communication. Their partition constraints depend on the represented state or operator.

Conserved Operator Subspaces. Bravyi et al. identify binary symmetries of fermionic Hamiltonians and remove qubits after selecting a symmetry sector [4]. West et al. describe a commutator graph whose connected components identify invariant operator subspaces under a set of generators [19]. Both methods establish algebraic conditions for independent evolution. *SectorQ* uses the ordinary binary inner product to construct labels invariant under generator XOR. These labels partition Pauli strings and are distinct from the symmetry eigenvalues used for qubit removal. A sector can also contain multiple connected components of the commutator graph. The system requires a partition conserved over a finite gate window and replaces it at the next boundary. This finite-window constraint permits nontrivial partitions even when all retained strings have identical values under the full circuit's conserved parities.

Fixed Linear Partitions. The partition label in paulistrings-rs is a binary matrix multiplied by a Pauli bit vector [12]. Its linear map identifies one communication partner for each nonlocal generator delta, and local coset execution processes independent groups of strings. The documented rows remain fixed during a run. Automatic row selection that considers balance and remote work is listed as an open problem. The parallel iQCC implementation similarly assigns Pauli words to processes through a bitmask and computes product destinations through XOR [6]. These mappings determine destinations even when a product requires remote communication. *SectorQ* additionally requires every selected row to annihilate every generator of the current window. The resulting sectors contain all contributions required for local merge and truncation throughout that window, with rows selected again at the next boundary.

Staged State Vector Simulation. Häner and Steiger group and schedule gates to reduce communication in a distributed simulation of 45 qubits [8]. Atlas partitions circuits into stages with an integer program, selects kernels with dynamic programming, and supports host-memory offloading [20]. These designs process a fixed state-vector index space across groups of gates. Pauli propagation has a variable string population because rotations create terms and merge and truncation operations remove them. A partition must preserve contributions to equal strings before truncation, while its load balance depends on the retained operator. *SectorQ* derives locality constraints from window generators and selects execution granularity from sampled terms. Host-memory execution uses these sectors as GPU tasks, and any subdivision must remain valid for the unfinished operations. The storage and scheduling mechanism consequently depends on the operator partition as well as the gate grouping.

5.2 Pauli Propagation Systems

Existing systems optimize propagation kernels, distributed storage, or repeated observable evaluations. These optimizations provide different execution and numerical conditions for comparison with *SectorQ*.

Distributed CPU Propagation. ORQA demonstrates propagation on Fugaku with up to one trillion Pauli strings and 2^{17} processes [5]. Its structured enumeration of string bits limits communication partners while maintaining balance. The distribution map determines remote updates as propagation advances through generators. MonoProp provides distributed Pauli and Majorana

propagation through a C++ implementation with Python interfaces [1]. Both systems support operator storage across CPU nodes. *SectorQ* uses distributed host memory for sectors and executes their windows through single-GPU propagation kernels. Its partition confines the window's equal-string contributions to one sector before GPU execution. The evaluation compares equal node counts with explicit CPU rank and thread configurations. The ORQA measurements use our reproduction, and MonoProp's different truncation rule requires separate interpretation of its timing and approximation error.

Propagation Libraries and GPU Kernels. Sparse Pauli dynamics and PauliPropagation.jl develop observable simulation and coefficient truncation methods [3, 17]. Quantinuum's SPD provides GPU execution, while Qiskit's Pauli propagation package supports Clifford operations, Pauli rotations, and noise channels [15, 16]. NVIDIA's *cuPauliProp* provides sparse Pauli operations on GPUs and a distributed mode [13]. *SectorQ* uses its single-GPU operations for rotations, merge, and truncation within a sector. The library determines the arithmetic, and the runtime determines the window partition, sector placement, and host-memory transfers. The evaluation compares the resulting system with original-circuit distributed *cuPauliProp* and separates Clifford preprocessing and deferred deduplication from sector execution. The single-GPU measurements also identify configurations where other libraries execute faster. The distributed mechanism addresses operator placement and merge locality beyond the execution of individual library operations.

Repeated Evaluations and Gradients. PADO-Pauli compiles circuits and observables into sparse matrices reused across batches of parameter values and supports gradients through PyTorch [9]. This compiled representation amortizes preparation over repeated evaluations during variational training. Backpropagating Pauli Propagation develops gradient evaluation with the computational complexity of forward propagation and reduces memory requirements through reversibility [11]. The reported *SectorQ* experiments measure one forward observable propagation for a specified circuit and coefficient cutoff. Its runtime chooses new conserved rows using the retained terms at successive windows. Batched evaluation and derivative computation introduce additional requirements for representation reuse and intermediate storage. Their costs require measurements that include the parameter batch or gradient computation, while the present evaluation measures distributed propagation time and capacity for one retained expansion.

6 Conclusion

We present *SectorQ*, a distributed Pauli propagation system that derives conserved parity sectors from the generators of each gate window and confines merge and truncation to sector execution within that window. The runtime selects conserved rows using sampled live terms during GPU execution and stores sectors in distributed host memory, with term redistribution at window boundaries and further sector subdivision when device buffers are insufficient. Across seven circuits, *SectorQ* reduces simulation time relative to original-circuit *cuPauliProp* by 4.4-9.7× at the largest common settings on 16 GPUs and completes a 140× larger term population on 512 GPUs.

References

[1] Algorithmiq. 2026. MonoProp: Classical Simulation through Pauli and Majorana Propagation. https://github.com/Algorithmiq/monoprop Software repository.

[2] Frank Arute et al. 2019. Quantum supremacy using a programmable superconducting processor. *Nature* 574 (2019), 505–510. doi:10.1038/s41586-019-1666-5

[3] Tomislav Begušić, Johnnie Gray, and Garnet Kin-Lic Chan. 2024. Fast and converged classical simulations of evidence for the utility of quantum computing before fault tolerance. *Science Advances* 10 (2024), eadk4321. doi:10.1126/sciadv.adk4321

[4] Sergey Bravyi, Jay M. Gambetta, Antonio Mezzacapo, and Kristan Temme. 2017. Tapering off qubits to simulate fermionic Hamiltonians. arXiv:1701.08213 https://arxiv.org/abs/1701.08213

[5] Lukas Broers, Rong-Yang Sun, and Seiji Yunoki. 2026. Scalable Simulation of Quantum Many-Body Dynamics with Or-Represented Quantum Algebra. *Physical Review Applied* 26 (2026), 024046. doi:10.1103/y8ft-m61w

[6] Scott N. Genin, Ohyun Kwon, Seyyed Mehdi Hosseini Jenab, Seon-Jeong Lim, Taehyung Kim, Tae-Gon Kim, Rami Gherib, Angela F. Harper, Ilya G. Ryabinkin, and Michael G. Helander. 2025. Towards Quantum Advantage in Chemistry. arXiv:2512.13657 https://arxiv.org/abs/2512.13657

[7] I. M. Georgescu, S. Ashhab, and Franco Nori. 2014. Quantum simulation. *Reviews of Modern Physics* 86 (2014), 153–185. doi:10.1103/RevModPhys.86.153

[8] Thomas Häner and Damian S. Steiger. 2017. 0.5 Petabyte Simulation of a 45-Qubit Quantum Circuit. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. Association for Computing Machinery, Article 33, 10 pages. doi:10.1145/3126908.3126947

[9] Hyunwoo Kim and Youngseok Lee. 2026. PADO-Pauli: A GPU-Accelerated Implementation of Pauli Propagation. arXiv:2610.17755 https://arxiv.org/abs/2610.17755

[10] Youngseok Kim, Andrew Eddins, Sajant Anand, Ken Xuan Wei, Ewout van den Berg, Sami Rosenblatt, Hasan Nayfeh, Yantao Wu, Michael Zaletel, Kristan Temme, and Abhinav Kandala. 2023. Evidence for the utility of quantum computing before fault tolerance. *Nature* 618 (2023), 500–505. doi:10.1038/s41586-023-06096-3

[11] Sheng-Hsuan Lin, Etienne Granet, Kevin Hémery, and Henrik Dreyer. 2026. Backpropagating Pauli Propagation. arXiv:2607.15184 https://arxiv.org/abs/2607.15184

[12] lkdvos. 2026. paulistrings-rs. https://github.com/lkdvos/paulistrings-rs/tree/99c34367f5cae3bb1f250fd4b1a56a8285f77e63 Software repository, commit 99c3436.

[13] NVIDIA. 2026. cuPauliProp: A High-Performance Library for Pauli Propagation Quantum Simulators. https://docs.nvidia.com/cuda/cuquantum/latest/cupauliprop/index.html Version 0.5.0 used in the evaluation.

[14] John Preskill. 2018. Quantum Computing in the NISQ era and beyond. *Quantum* 2 (2018), 79. doi:10.22331/q-2018-08-06-79

[15] Qiskit contributors. 2026. Pauli Propagation. https://github.com/Qiskit/pauli-prop Software repository.

[16] Quantinuum. 2026. SPD: Sparse Pauli Dynamics. https://github.com/Quantinuum/spd Software repository.

[17] Manuel S. Rudolph, Tyson Jones, Yanting Teng, Armando Angrisani, and Zoë Holmes. 2026. Pauli Propagation: A Computational Framework for Simulating Quantum Systems. *PRX Quantum* 7 (2026), 032001. doi:10.1103/6vd7-l9bn

[18] Joseph Tindall, Matthew Fishman, E. Miles Stoudenmire, and Dries Sels. 2024. Efficient Tensor Network Simulation of IBM's Eagle Kicked Ising Experiment. *PRX Quantum* 5 (2024), 010308. doi:10.1103/PRXQuantum.5.010308

[19] Maxwell West, Neil Dowling, Angus Southwell, Martin Sevior, Muhammad Usman, Kavan Modi, and Thomas Quella. 2025. A graph-theoretic approach to chaos and complexity in quantum systems. *SciPost Physics Core* 8 (2025), 081. doi:10.21468/SciPostPhysCore.8.4.081

[20] Mingkuan Xu, Shiyi Cao, Xupeng Miao, Umut A. Acar, and Zhihao Jia. 2024. Atlas: Hierarchical Partitioning for Quantum Circuit Simulation on GPUs. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 17 pages. https://www.pdl.cmu.edu/ftp/associated/Xu-AtlasSC24.pdf